

第3章 软件构建-2

- 3.1编码概述
 - 3.2编程语言和编码规范
 - 3.3代码版本控制
 - 3.4可以工作的类
 - 3.5高质量的子程序
 - 3.6防御式编程
 - 3.7变量和语句的构建
-

3.4 可以工作的类

- 类是由一组数据和子程序构成的集合，这些数据和子程序共同拥有一组内聚的、明确定义的职责。类也可以只是由一组子程序构成的集合，这些子程序提供了一组内聚的服务，哪怕其中并未设计公用的数据。

类的基础：抽象数据类型（ADT）

- **抽象数据类型**是指一些数据以及对这些数据所进行的操作的集合。
- 使用ADT的益处：**可以隐藏实现细节**、改动不会影响到整个程序、让接口提供更多信息、更容易提高性能、让程序的正确性更显而易见、程序更具自我说明性、无需在程序内到处传递数据、可以像在现实世界中那样操作实体，而不用在底层实现上操作它。

例子：文件类（隐藏了磁盘的读写细节）

类的基础：抽象数据类型（**ADT**）

- **举例。**一个程序能用不同的字体、字号和文字属性（如粗体和斜体等）来控制显示在屏幕上的字体。

不用**ADT**：

`currentFont.size = 16` (大小设为16个像素)

`currentFont.attr = currentFont.attr or 0x02` (设为粗体)

使用**ADT**：

`currentFont.setSizeInPoints(sizeInPoints)`

`currentFont.setSizeInPixels(sizeInPixels)`

`currentFont.setBoldOn()`

`currentFont.setBoldOff()`

`currentFont.setItalicOn()`

`currentFont.setItalicOff()`

良好的类接口

- 第一步也是最重要的一步就是创建一个良好的类接口。一个没有经过良好抽象的类可能会包含大量混杂的函数。

```
public class Stu {  
    private String id;  
    private String name;  
    private Int age;  
    public getX() { }  
    public setXx() { }  
    public void study() { }  
    public void initializeGlobalDate() { } //不相关的函数  
    public void printReport(Report report) { } //不相关的函数  
}
```

良好的类接口

为了追求设计优秀，创建类的好的抽象接口的指导建议包括：

- 类的接口应展现**一致的抽象层次**

在下面这个例子中，类的接口不够协调，因为它的抽象层次不一致：



**CODING
HORROR**

这些子程序的抽象在
“雇员”这一层次上。

这些子程序的抽象在
“列表”这一层次上。

C++示例：混合了不同层次抽象的类接口

```
class EmployeeCensus: public ListContainer {
public:
    ...
    // public routines
    void AddEmployee( Employee employee );
    void RemoveEmployee( Employee employee );

    Employee NextItemInList();
    Employee FirstItem();
    Employee LastItem();
    ...
private:
    ...
};
```

C++示例：有着一致抽象层次的类接口

```
class EmployeeCensus {  
public:  
    ...  
    // public routines  
    void AddEmployee( Employee employee );  
    void RemoveEmployee( Employee employee );  
    Employee NextEmployee();  
    Employee FirstEmployee();  
    Employee LastEmployee();  
    ...  
private:  
    ListContainer m_EmployeeList;  
    ...  
};
```

所有这些子程序的抽象现在都是在“雇员”这一层次上了。

使用 ListContainer 库这一实现细节现在已经被隐藏起来了。

良好的类接口（创建类的好的抽象接口的指导建议）

- 一定要理解类所实现的抽象是什么；
 - 电子表格（150子程序）、网格（15子程序）
- 提供成对的服务；
 - 一个操作开灯，另一个操作关灯
 - 一个操作激活菜单项，另一个操作屏蔽菜单项
- 把不相关的信息转移到其他类中；
 - 某个类中一半子程序使用着该类的一半数据，而另一半子程序使用另一半数据。需要拆开。
- 谨防在修改时破坏接口的抽象；
- 不要添加与接口不一致的公用成员；
- 同时考虑抽象性和内聚性。
 - 有很强内聚性的类，抽象也会好。如果发现内聚性很弱，想想是否表现为一致的抽象

良好的类接口（良好的封装的指导建议）

- 封装是一个比抽象更强的概念。抽象通过一个可以忽略显示细节的模型来管理复杂度，封装则强制阻止看到细节。
- 尽可能地限制类和成员的可访问性；
 - `private`
- 不要公开暴露成员数据；
- 避免把私用的实现细节放入类的接口中；
- 不要对类的使用者做出任何假设；
 - 请把 `x`, `y` 和 `z` 初始化为 `1.0`，因为如果把它们
 - 初始化为 `0.0` 的话，`DerivedClass` 就会崩溃。

良好的类接口（良好的封装的指导建议）

- 避免使用友元类。（C++有）
- 让阅读代码比写代码更方便。阅读代码的次数要比编写代码多得多，即使在开发的初期也是如此。因此，为了让编写代码更方便而降低代码的可读性是非常不经济的。
- 要格外警惕从语义上破坏封装性。例如：
 - 不去调用 A 类的 `InitializeOperations()` 子程序，因为你知道 A 类的 `PerformFirstOperation()` 子程序会自动调用它。
 - 不在调用 `employee.Retrieve(database)` 之前去调用 `database.Connect()` 子程序，因为你知道在未建立数据库连接的时候 `employee.Retrieve()` 会去连接数据库的。

这个例子问题在于让调用方代码依赖于内的私有实现。

3.4 可以工作的类

有关设计和实现的问题

- 通过包含来实现“有一个”的关系；
- 通过继承来实现“是一个”的关系
- 警惕有超过约7个数据成员的类（研究表明，人们在做其他事情时能记住的离散项目的个数是 7 ± 2 ，Miller法则）

包含是一个非常简单的概念，它表示一个类含有一个基本数据元素或对象。与包含相比，关于继承的论述要多得多，这是因为继承需要更多的技巧，而且更容易出错，而不是因为继承要比包含更好。包含才是面向对象编程中的主力技术。

通过包含来实现“有一个/has a”的关系，可以把包含想象成“有一个”关系。比如说，一名雇员“有一个”姓名，“有一个”电话号码”等。通常，你可以让姓名、电话号码成为 **Employee** 类的数据成员，从而建立这种关系。

通过继承来实现“是一个”的关系。“老虎”是一个“动物”，“老虎”类继承“动物”类；“大雁”是一个“鸟”，“大雁”类继承“鸟”类。

继承的概念是说一个类是另一个类的特化(specialization)。继承的目的在于通过“定义能为两个或更多派生类提供共有元素的基类”的方式写出更精简的代码。其中的共有元素可以是子程序接口、内部实现、数据成员等。继承能把这些共有的元素集中在一个基类中，从而有助于避免在多处出现重复的代码和数据。

当决定使用继承时，你必须要做如下几项决策。

- 对于基类的每个成员函数而言，它应该对派生类可见吗？它应该有默认的实现吗？这一个默认的实现能被覆盖吗？
- 对于基类的每一个数据成员而言（包括变量、常量、枚举等），它应该对派生类可见吗？

成员函数和数据成员（指导建议）

- 让类中子程序(方法)的数量尽可能少；
 - 类中子程序越多，出错率越高
- 禁止赋值、禁止某个对象被构造、禁止某个函数被调用，采用 **private**；
- 减少类所调用的不同子程序的数量；
 - 类用到其他类的数量越高，出错率越高一扇出
- 对其他类的子程序的间接调用要尽可能少；
 - **A**对象可以任意调用它自己的所有子程序。如果**A**对象创建了一个**B**对象，它也可以调用**B**对象的任何（公共）子程序，但是它不应该避免再调用**B**对象所提供的对象中的子程序
- 一般来说，应尽量减小类和类之间相互合作的范围。
- 应该在构造函数中初始化所有数据成员。

创建类的原因

- 为实现世界中的对象建模；
- 为抽象的对象建模；
 - Shape（形状）---Circle（圆）、Square（正方形），如何建Shape？
- 降低复杂度；隐藏实现细节；
 - 类创建好后，忘记细节；
- 限制变动的影响范围；
- 隐藏全局数据；

创建类的原因

- 让参数传递更顺畅；

如果你需要把一个参数在多个子程序之间传递，这有可能表明应该把这些子程序重构到一个类里，把这个参数当做对象数据（成员变量）来共享。

- 建立中心控制点；
- 让代码更易于重用；
- 为程序族做计划；
- 把相关操作包装到一起；
- 实现某种特定的重构。

建立中心控制点。在一个地方控制一项任务是个好主意。控制可以表现为很多形式。了解一张表中记录的数目是一种形式；对文件、数据库连接、打印机等设备进行的控制又是一种。用一个类来读写数据库则是集中控制的又一种形式。如果需要把数据库转换为平坦的文件或内存数据，有关改动也只会影响一个类。

为程序族做计划。如果你预计到某个程序会修改，你可以把预计要被改动的部分放到单独的类里，同其他部分隔离开，这是个好主意。之后你就可以只修改这个类或用新的类来取代它，而不会影响到程序的其余部分了。仔细考虑整个程序族（**family of programs**）的可能情况，而不单是考虑单一程序的可能情况。

应该避免的类

- 避免创建万能类；
- 消除无关紧要的类；
 - 如果一个类只包含数据不包含操作？
 - 降级，让属性成为另一个类的属性
- 避免用动词命名的类。比如：
DatabaseInitialization（数据库初始化类）不好。
 - 只有行为没有数据的类不是一个真正的子类
 - 降级，成为另一个类的子程序。

CHECKLIST: Class Quality

核对表：类的质量

抽象数据类型

- ☐ 你是否把程序中的类都看做是抽象数据类型了？是否从这个角度评估它们的接口了？

抽象

- ☐ 类是否有一个中心目的？
- ☐ 类的命名是否恰当？其名字是否表达了其中心目的？
- ☐ 类的接口是否展现了一致的抽象？
- ☐ 类的接口是否能让人清楚明白地知道该如何用它？
- ☐ 类的接口是否足够抽象，使你能不必顾虑它是如何实现其服务的？你能把类看做黑盒子吗？
- ☐ 类提供的服务是否足够完整，能让其他类无须动用其内部数据？
- ☐ 是否已从类中除去无关信息？
- ☐ 是否考虑过把类进一步分解为组件类？是否已尽可能将其分解？
- ☐ 在修改类时是否维持了其接口的完整性？

封装

- ☐ 是否把类的成员的可访问性降到最小？
- ☐ 是否避免暴露类中的数据成员？
- ☐ 在编程语言所许可的范围内，类是否已尽可能地对其他类隐藏了自己的实现细节？
- ☐ 类是否避免对其使用者，包括其派生类会如何使用它做了假设？
- ☐ 类是否不依赖于其他类？它是松散耦合的吗？

继承

- ☐ 继承是否只用来建立“是一个/is a”的关系？也就是说，派生类是否遵循了 LSP（Liskov 替换原则）？
- ☐ 类的文档中是否记述了其继承策略？
- ☐ 派生类是否避免了“覆盖”不可覆盖的方法？
- ☐ 是否把公用的接口、数据和行为都放到尽可能高的继承层次中了？
- ☐ 继承层次是否很浅？
- ☐ 基类中所有的数据成员是否都被定义为 `private` 而非 `protected` 的了？

跟实现相关的其他问题

- ☐ 类中是否只有大约七个或更少的数据成员？
- ☐ 是否把类直接或间接调用其他类的子程序的数量减到最少了？
- ☐ 类是否只在绝对必要时才与其他的类相互协作？
- ☐ 是否在构造函数中初始化了所有的数据成员？
- ☐ 除非拥有经过测量的、创建浅层复本的理由，类是否都被设计为当作深层复本使用？

第3章 软件构建

- 3.1编码概述
 - 3.2编程语言和编码规范
 - 3.3代码版本控制
 - 3.4可以工作的类
 - 3.5高质量的子程序
 - 3.6防御式编程
 - 3.7变量和语句的构建
-

3.5高质量的子程序

- 子程序 是为实现一个特定的目的而编写的一个可被调用的方法或过程
- 子程序的使用使得程序变得更加易读，更易于理解。

子程序

这个子程序里有哪些不妥呢？

C++示例：低质量的子程序

```
void HandleStuff( CORP_DATA & inputRec, int crntQtr, EMP_DATA empRec,  
    double & estimRevenue, double ytdRevenue, int screenX, int screenY,  
    COLOR_TYPE & newColor, COLOR_TYPE & prevColor, StatusType & status,  
    int expenseType )  
{  
    int i;  
    for ( i = 0; i < 100; i++ ) {  
        inputRec.revenue[i] = 0;  
        inputRec.expense[i] = corpExpense[ crntQtr ][ i ];  
    }  
    UpdateCorpDatabase( empRec );  
    estimRevenue = ytdRevenue * 4.0 / (double) crntQtr;  
    newColor = prevColor;  
    status = SUCCESS;  
    if ( expenseType == 1 ) {  
        for ( i = 0; i < 12; i++ )  
            profit[i] = revenue[i] - expense.type1[i];  
    }  
    else if ( expenseType == 2 ) {  
        profit[i] = revenue[i] - expense.type2[i];  
    }  
    else if ( expenseType == 3 ) {  
        profit[i] = revenue[i] - expense.type3[i];  
    }  
}
```

缺少注释

函数目的？

100等数值含义等

子程序

- 名字？没有说明程序功能
- 文档？提高可移植性
- 布局？随意，如

`expenseType==2` 和 `expenseType==3` 这两处的代码风格

- 输入变量值被改变？`inputRec`
- 没有一个单一目的？这个子程序初始化了一些变量，向数据库写入数据，做了一些计算。这几件事看不出有任何联系
- 没有防范错误数据处理 如果 `crntQtr` 等于 0
- 使用神秘数值：100、4.0、12、3、2
- `screenx`、`screeny` 没有被使用
- 一个参数传递方式有误：`preColor` 标为引用但没对它赋值
- 参数太多
- 参数顺序混乱（应按照输入-修改-输出的顺序排列参数）

创建子程序的理由

■ 降低复杂度

- 通过创建子程序来隐藏一些信息，一旦程序写好了就忘记这些细节，直接调用该子程序而无须了解其内部工作细节。
- 通过子程序对代码的实现隐藏细节，进行一次抽象，将复杂度降低一级

当内部循环或条件判断的嵌套层次很深时，就意味着需要从子程序提取出新的子程序了。把嵌套的部分提取出来形成一个独立的子程序，可以降低外围子程序的复杂度。

创建子程序的理由

- 引入中间、易懂的抽象

```
if ( node <> NULL ) then
  while ( node.next <> NULL ) do
    node = node.next
    leafName = node.name
  end while
else
  leafName = ""
end if
```

读懂下面这条语句就更容易：

```
leafName = GetLeafName( node )
```

创建子程序的理由

■ 避免代码重复

- 把相同的代码放入新的子程序中，再让其余的代码来调用这个子程序，与代码的重复出现相比，让相同的代码只出现一次可以节约空间。
- 代码改动起来更方便，只需要在一处修改即可，代码也会更可靠。

创建子程序的理由

- 隐藏顺序
 - 把处理事件的顺序隐藏起来。
- 提高可移植性
 - 可以用子程序来隔离代码中不可移植的部分，从而明确识别和隔离未来的移植工作

不可移植的部分包括编程语言所提供的非标准功能、对硬件的依赖，以及对操作系统的依赖等。

创建子程序的理由

■ 改善性能

- 通过使用子程序，把代码集中在一处可以更方便地查出哪些代码的运行效率低下，同时，在一处优化代码，就能使用到该子程序的所有代码都从中受益
- 把代码集中在一处后，想用更高效的算法或更快速高效的语言来重写代码也更容易

■ 提高可读性

- 提高可读性，甚至达到自我注解的地步

在子程序层上设计

■ 内聚性与耦合性

- **耦合**是衡量**不同**模块彼此间互相依赖连接的紧密程度
- **内聚**是衡量**一个**模块内部各个元素彼此结合的紧密程度。
- 对于程序来说，内聚性是指子程序中各种操作之间联系的紧密程度，我们的目标是让每一个子程序只把一件事做好，不再做其他事情。
- 一项研究表明，耦合度与内聚性之比最高的那些子程序，其中所含的错误是耦合度与内聚性最低的子程序的7倍之多，而其修正成本则为20倍。

在子程序层上设计

■ 内聚性与耦合性



- **数据耦合**：是指两个模块彼此交换数据。例如：
：一个模块的输出数据另一个模块的输入数据，
或一个模块带参数调用另一个模块。
- **内容耦合**：是耦合程度最高的一种形式。若一个模块直接访问另一个模块的内部代码或数据，
则会出现内容耦合。
- **尽量使用数据耦合，少用控制耦合，限制公共耦合的范围，完全不用内容耦合。**

模块间内聚的类型



- **功能内聚**：是内聚度最高的一种模块类型。如果模块仅完成一个单一的功能，且该模块的所有部分是实现这一功能所必需的，没有多余的语句，则该模块为功能内聚型。
- **偶然内聚**：是指模块内的各个部分(语句)之间不存在有意义的联系，它们之所以能构成一个模块完全是偶然的原因。
- **在模块设计时应力争做到高内聚，并且能够辨别出低内聚的模块，加以修改使之提高内聚度并降低模块间的耦合度。**

好的子程序名

好的子程序名字能清晰描述子程序所做的一切, 命名原则:

- 描述子程序所做的所有事情

子程序的名字应当描述其所有的输出结果

`addStudent`、`delScore`、`updateBook`

- 避免使用无意义的、模糊或表述不清的动词

命名的描述要精确, 不要使用一些泛泛的动词, 最好使用过程描述。`HandleOutput()`改为`FormatAndPrintOutput()`

- 绝对不要通过数字来形成不同的子程序名字

不要将功能相近的函数命名为 `part1`, `part2`这一类名字

- 根据需要确定子程序名字的长度

给子程序命名的重点是尽可能含义清晰, 也就是说, 子程序名的长短要视该名字是否清晰易懂而定

好的子程序名

■ 给子程序起名时使用语气强烈的动词加宾语的形式

一个具有功能内聚性的过程通常是针对一个对象执行一种操作，过程的名字应该能反映该过程所做的事情，而一个针对某对象执行的操作就需要一个动词+宾语形式的名字

■ 准确使用对仗词

add/remove

begin/end

create/destroy

first/last

get/put

get/set

increment/decrement

insert/delete

lock/unlock

min/max

next/previous

old/new

open/close

show/hide

source/target

start/stop

up/down

如何使用子程序的参数

■ 不要把子程序的参数用作工作变量

Java示例：不恰当地使用输入参数

```
int Sample( int inputVal ) {  
    inputVal = inputVal * CurrentMultiplier( inputVal );  
    inputVal = inputVal + CurrentAdder( inputVal );  
    ...  
    return inputVal;  
}
```

此处inputVal已经不再是当初传入的值了。

Java示例：正确地使用输入参数

```
int Sample( int inputVal ) {  
    int workingVal = inputVal;  
    workingVal = workingVal * CurrentMultiplier( workingVal );  
    workingVal = workingVal + CurrentAdder( workingVal );  
    ...  
    return workingVal;  
}
```

如果在此处或其他地方还可以使用inputVal最初传入的值。

如何使用子程序的参数

■ 在接口中对参数的假定加以说明

- 参数是仅用于输入的、要被修改的、还是仅用于输出的；
- 表示数量的参数的单位（英寸、英尺、米等）；
- 如果没有用枚举类型的话，应说明状态代码和错误值的含义；
- 所能接受的数值的范围；
- 不该出现的特定数值。

■ 程序的参数个数最好小于7个

如何使用子程序的参数

■ 为子程序传递用以维持其接口抽象的变量或对象

有一个对象，它有 10 个数据，被调用的子程序只需要其中的 3 项数据就能进行操作。究竟是应该传递仅需的 3 项数据，还是直接传递整个对象？

如果要表达的抽象是子程序期望3项特定的数据，但这3项数据只是碰巧由同一个对象所提供，那就应该单独传递这3项数据。然而，如果子程序接口想表达的抽象是想一直拥有某个特定对象，且子程序要对这一对象执行各种的操作，如果仅传递3项数据的话，就是破坏了接口的抽象，那就应该传递对象。

CHECKLIST: High-Quality Routines

核对表：高质量的子程序

大局事项

- ☐ 创建子程序的理由充分吗？
- ☐ 一个子程序中所有适于单独提出的部分是不是已经被提出到单独的子程序中了？
- ☐ 过程的名字中是否用了强烈、清晰的“动词+宾语”词组？函数的名字是否描述了其返回值？
- ☐ 子程序的名字是否描述了它所做的全部事情？
- ☐ 是否给常用的操作建立了命名规则？
- ☐ 子程序是否具有强烈的功能上的内聚性？即它是否做且只做一件事，并且把它做得很好？
- ☐ 子程序之间是否有较松的耦合？子程序与其他子程序之间的连接是否是小的（small）、明确的（intimate）、可见的（viaible）和灵活的（flexible）？
- ☐ 子程序的长度是否是由其功能和逻辑自然确定，而非遵循任何人为的编码标准？

参数传递事宜

- ❑ 整体来看,子程序的参数表是否表现出一种具有整体性且一致的接口抽象?
- ❑ 子程序参数的排列顺序是否合理? 是否与类似的子程序的参数排列顺序相符?
- ❑ 接口假定是否已在文档中说明?
- ❑ 子程序的参数个数是否没超过 7 个?
- ❑ 是否用到了每一个输入参数?
- ❑ 是否用到了每一个输出参数?
- ❑ 子程序是否避免了把输入参数用做工作变量?
- ❑ 如果子程序是一个函数,那么它是否在所有可能的情况下都能返回一个合法的值?

第3章 软件构建

- 3.1编码概述
 - 3.2编程语言和编码规范
 - 3.3代码版本控制
 - 3.4可以工作的类
 - 3.5高质量的子程序
 - **3.6防御式编程**
 - 3.7变量和语句的构建
-

3.6 防御式编程

- 防御式编程概念
- 断言
- 错误处理技术
- 异常
- 隔离程序(隔栏)
- 辅助调试代码

3.6.1 防御式编程概念

防御式编程的概念来自于**防御式驾驶**。在防御式驾驶中要建立这样一种思维：你永远不能确定另一位司机将要做什么。这样才能确保在其他人做出危险动作是你也不会受到伤害。



如何做到防御式驾驶？

3.6.1 防御式编程概念

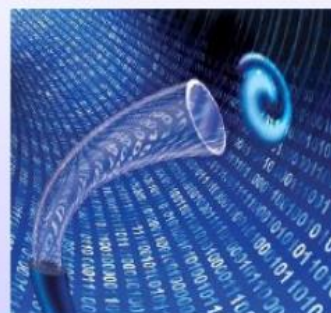
- 防御式编程的主要思想是：子程序不因传入错误数据而被破坏，哪怕是由其他子程序产生的错误数据。
- 其核心是承认程序都会有问题，都需要被修改，这是保护的基础。
- 防御式编程主要用来保护程序免遭非法输入数据的破坏

保护程序免遭非法输入数据的破坏

- 仅仅“垃圾进，垃圾出”还不够。不管进来什么，好的程序都不会生成垃圾，而是做到：
 - “垃圾进，什么都不出”
 - “进来垃圾，出去是出错提示”
 - “不许垃圾进来”
- 通常有三种方法来处理进来垃圾（非法数据）的情况：
 - 检查所有来源于外部数据的值
 - 检查子程序所有输入参数的值
 - 决定如何处理错误的输入数据

保护程序免遭非法输入数据的破坏

- 检查所有来源于外部数据的值
 - 对于数值，须确保它在可接受范围内；
 - 对于字符串，须确保它的长度没有超长等。
- 外部数据来源于文件，用户输入，网络或其它外部接口，应检查这些值是否在合法的范围之内



外部数据源

保护程序免遭非法输入数据的破坏

- 检查子程序所有输入参数的值
 - 数据来自其他子程序。

- ◆ 子程序参数可能是错误的，需要检查
- ◆ 例如

```
void Get_Process_Type(short shNo)
{
    if ( ( 0<=shNo ) && ( shNo < MAX_CHA_NUM ) )
    {
        return ( Win2[shNo].shType );
    }
}
```

保护程序免遭非法输入数据的破坏

- 决定如何处理错误的输入数据
 - 需要根据情况的不同，来确定不同的解决方案。
- 主要的防御式编程技术有断言、错误处理技术、异常、隔栏、辅助调试代码

防范看似微小的错误，收获可能远远超出你的想象



西雅图90号州际浮桥的一部分在一场风暴中沉没了，原因是未遮盖浮箱，而在风暴中进水，使得桥体过重而无法继续漂浮。在建设时要防范一些小事情，它们的严重性往往会超过你的预期

3.6.2 断言

- 断言 (assertion) 是指在开发期间使用的、让程序在运行时进行自检的代码（通常是一个子程序或宏）。
- 断言为真，则表明程序运行正常，而断言为假，则意味着它已经在代码中发现了意料之外的错误。
- 关键点：它是用来处理“绝不应该”发生的状况。
- 避免把需要执行的代码放到断言中如果把代码写在断言里，那么当你关闭断言功能时，编译器很可能就把这些代码除排在外了。

断言语法

语法

1、assert(boolean表达式)

如果<boolean表达式>为true，则程序继续执行。

如果为false，则程序抛出AssertionError，并终止执行。

2、assert(boolean表达式) : <错误信息表达式>

如果<boolean表达式>为true，则程序继续执行。

如果为false，则程序抛出java.lang.AssertionError，并输入<错误信息表达式>。

断言例子：

```
public void fun1(String strNo, double score) {  
    assert strNo != NULL: “学号不能为空” ;  
    assert score >= 0 && score <= 100: “分数不能为负  
或大于100分” ;  
    System.out.println(“断言没有问题, Go! ”);  
    //next, do something else  
    System.out.println(strNo);  
    .....  
}
```


在下面这个例子中，就使用了断言来说明 Velocity（速度）子程序的前条件和后条件：

Visual Basic示例：使用断言来记述前条件和后条件

```
Private Function Velocity ( _  
    ByVal latitude As Single, _  
    ByVal longitude As Single, _  
    ByVal elevation As Single _  
    ) As Single  
  
    ' Preconditions  
    Debug.Assert ( -90 <= latitude And latitude <= 90 )  
    Debug.Assert ( 0 <= longitude And longitude < 360 )  
    Debug.Assert ( -500 <= elevation And elevation <= 75000 )1  
    ...  
  
    ' Postconditions  
    Debug.Assert ( 0 <= returnVelocity And returnVelocity <= 600 )  
  
    ' return value  
    Velocity = returnVelocity  
End Function
```


断言能够检查到的错误

1. 输入参数或输出参数取值处于预期范围内
2. 子程序开始（或结束）时文件或流是处于打开（或关闭）状态
3. 子程序开始（或结束）时文件或流的读写位置位于开头（或结尾）处
4. 文件或流已用只读、只写或读写方式打开
5. 仅用于输入的变量的值没有被子程序所修改
6. 指针非空
7. 传入子程序的数组或其他容器至少能容纳X个数据元素
8. 表已经初始化，存储着真实的数据

什么时候使用断言

- ◆ 断言主要用于开发和维护阶段
- ◆ 断言只在开发阶段被编译到目标代码中，而在生成产品代码时不编译到产品代码中

断言和错误处理的使用状况

- ◆ **断言(assertion)**用来检查永远不应该发生的状况，断言用于检查代码中的bug，因此断言通常采用**强制退出**程序的方式
- ◆ **错误处理 (error-handling code)** 用来检查程序的非正常状况，这些情况能在写代码时就预料到，比如从文件读取的汽车速度数据，应该在0-300Km/h范围内，超出了这个范围被认为数据出错

3.6.3 错误处理技术

错误处理技术包括以下几种：

■ 返回中立值

- 继续执行程序操作，返回一个中立的值。比如整形数据返回0，字符串数据返回null等。

■ 换用下一个正确的值

- 使用下一个正确的数值来替换，使得程序的继续执行运行下去。

■ 返回与前次相同的数据

- 使用前一个正确的数值来替换，使得程序的继续执行运行下去。

■ 换用最接近的合法值

- 预计正确的数值应该满足一个可接受的数值范围的。当得到的数据不在范围之内的时候，选择距离正确数值范围最近的值作为结果

- 把警告信息记录到日志文件中
- 返回一个错误码
- 调用错误处理子程序或对象
 - 当发现非法数据的时候，通过调用项目中专门用于处理错误异常的子程序来处理。
- 当错误发生是显示出错消息
- 用最稳妥的方法在局部处理错误
 - 发现错误的时候，尽量放在子程序内部完成处理。
- 关闭程序
 - 虽然很多时候，对非法数据的处理就是在避免程序的关闭。但是一些特殊的行业里面，检测到数据的时候，必须关闭。比如航天，比如医药制造。

错误处理例子：

```
if( ( shChannel_No < 0 ) || ( shChannel_No >
    MAX_CHANNEL_NUMBER )
{
    shChannel_No = 0;           ///返回中立值
}

if( iHeart_Rate > MAX_HEART_RATE )
{
    iHeart_Rate = iLast_Heart_Rate;    ///返回上一次的值
}

if( ( iVelocity > MAX_VELOCITY )
{
    iVelocity = DEF_VELOCITY;         ///返回默认值
}
```

错误处理

■ 健壮性和正确性

- 正确性：永远不能返回错误的结果，哪怕程序的强制停止。
- 健壮性：不断的尝试某些措施，以保证程序的正常运行，哪怕返回的数值是错误的。

- 在处理错误的时候，需要根据不同的行业软件来选择追求正确性还是追求健壮性。如消费类软件追求健壮性。
- 处理错误的方法有很多，但是当项目中选中某一个方法的时候，那么就要确保整个项目采用统一的处理方案。
- 构建一个集中的错误处理程序，所有错误都由其完成。

断言和错误处理技术区别

- 对于高健壮性的代码，应该先使用断言再处理错误。对于每种可能出错的条件，通常子程序要么使用断言，要么使用错误处理代码来进行处理，但是不会同时使用二者。

区别

- 断言技术用来检查永远不应该发生的状况，检查代码的bug；
- 错误处理技术用来检查程序的非正常状况，用于处理能够预测的数据出错；
- 断言代码，一般在生成产品代码时不编译到产品代码中
- 错误处理代码，一般在生成产品代码时要编译到产品代码中

3.6.4 异常

- ◆ 异常（Exceptions）是把代码中的错误或异常事件传递给调用方代码的一种特殊手段
- ◆ 异常的基本结构是：子程序使用throw抛出一个异常对象，再被调用链上层其他子程序的try-catch语句捕获

使用异常的建议

1. 用异常通知程序的其它部分，发生了不可忽略的错误
2. 只在真正例外的情况下抛出异常
3. 不能用异常来推卸责任
4. 避免在构造函数或析构函数中抛出异常
5. 在恰当的抽象层次抛出异常
6. 在异常消息中加入关于异常发生的全部信息
7. 避免使用空的catch语句
8. 考虑创建一个集中的异常报告机制
9. 考虑异常的替换方案

避免使用空的Catch语句

◆ 空的Catch语句意味着程序存在问题

Java示例：忽略异常的错误做法

```
try {  
    ...  
    // Lots of code  
    ...  
} catch ( AnException exception ) {  
}
```

Java示例：忽略异常的正确做法

```
try {  
    ...  
    // Lots of code  
    ...  
} catch ( AnException exception ) {  
    LogError( " Unexpected exception " );  
}
```

考虑创建集中的异常报告机制

- 构建自己的异常报告函数

Visual Basic示例：集中的异常报告机制（第一部分）

```
Sub ReportException ( _  
    ByVal className, _  
    ByVal thisException As Exception, _  
)  
    Dim message As String  
    Dim caption As String  
  
    message = _  
        " Exception: " & thisException.Message & "." & ControlChars.CrLf & _  
        " Class: " & className & ControlChars.CrLf & _  
        " Routine: " & thisException.TargetSite.Name & ControlChars.CrLf & _  
    caption = " Exception "  
    MessageBox.Show ( message, caption, MessageBoxButtons.OK , _  
        MessageBoxIcon.Exclamation )  
End Sub
```


考虑异常的替换方案

- ◆ 异常的替换方法包括：
 1. 在局部处理错误
 2. 使用错误码传递错误
 3. 在日志中记录异常信息
 4. 关闭系统等

3.6.5 隔离程序(隔栏)

- ◆ 隔栏 (**barricade**) 是一种容损策略，将程序的外部 and 内部进行隔离
- ◆ 把某些接口选定为安全区域的边界，对穿越安全区域边界的数据进行合法性校验，并当数据非法时进行处理

隔离程序

- 把某些接口选定为“安全”区域的边界。对穿越安全区域边界的数据进行合法性校验，并当数据非法时做出敏锐的反应。
- 同样可以在类的层次采用这种方法。类的公用方法可以假设数据是不安全的，它们要负责检查数据并进行清理。一旦类的公用方法接受了数据，那么类的私用方法就可以假定数据都是安全的了

3.6.5 隔离程序(隔栏)

程序的外部 and 内部被明显隔离

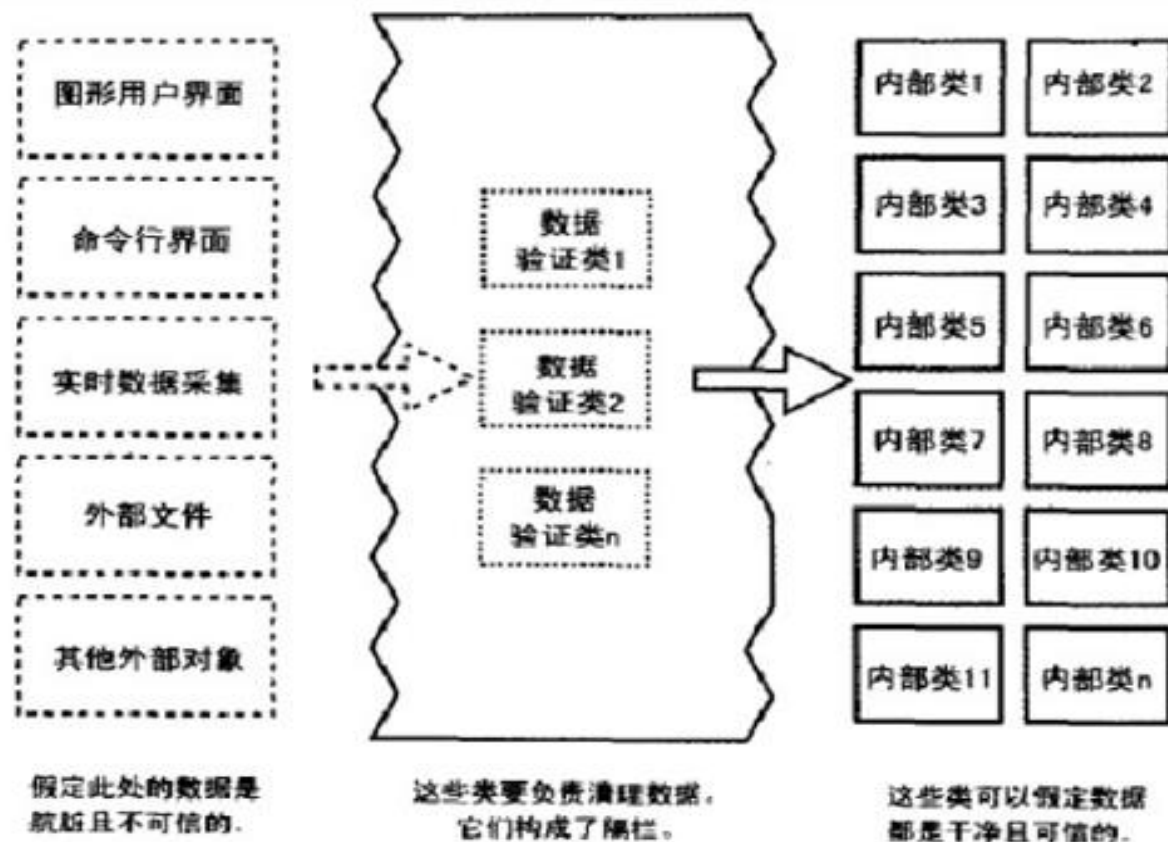


图8-2 让软件的某些部分处理“不干净的”数据，而让另一些部分处理“干净的”数据，即可让大部分代码无须再担负检查错误数据的职责

隔栏与断言、错误处理技术的关系

- ◆ 隔栏的使用使断言和错误处理有了清晰的区分：
 1. 隔栏外部的程序使用错误处理技术，外部的数据是不安全的
 2. 隔栏内部的程序就应该使用断言技术因为传入程序内部的数据都已经过了隔栏的处理，应该是正确的，如果出错，则说明程序本身出错

3.6.6 辅助调试代码

- 防御式编程的一个重要方面就是使用辅助调试代码（调试助手），以帮助快速地检测错误。
- 应尽早引入辅助调试的代码，帮助程序员在产品的开发期找出错误。

要想在编译时包含调试用的代码，使用`#define`来定义`DEBUG`符号。要想将其排除在外，则不要定义`DEBUG`符号。

C++示例：直接使用预处理器来控制调试用的代码

```
#define DEBUG
...

#if defined( DEBUG )
// debugging code
...

#endif
```

区分产品调试版和发布版

- ◆ 通常而言，产品发布版要求产品性能良好，节约资源，因此通常不含有辅助调试代码
- ◆ 而在产品调试版中，可以不考虑性能和资源问题，可以在代码中加入各种辅助调试代码，以帮助程序员容易地找到程序中可能存在的问题

辅助调试代码

- 不要自动地把开发版的限制强加于之产品版上
 - 产品版本和开发版本，两者的限制要求是不同的。产品级的软件要求能够快速的运行，而开发中的软件允许运行缓慢。
- 尽早引入辅助调试的代码
- 采用进攻式编程
 - 什么是进攻式编程？开发阶段勇敢的面对各种异常，产品发布的时候，使之拥有自我修复的功能。这样的做法的好处是，能够在开发阶段遭遇更多的问题，以致于在产品发布之后，产生的问题会相对较少。
- 计划移除调试辅助的代码

保留防御性代码的建议

1. 保留那些检查**重要错误**的代码
2. 去掉检查**细微错误**的代码
3. 去掉可以导致程序**硬性崩溃**的代码
4. 保留可以让程序**稳妥崩溃**的代码
5. 为你的技术支持人员**记录错误信息**
6. 确认留在代码中的**错误消息是友好的**

对防御式编程采取防御式姿态

- ◆ 过度的防御式编程也会引入问题
- ◆ 如果在程序的每一个想到的地方都进行参数检查，错误保护等，那么程序将变得**臃肿而缓慢**，更糟糕的是，防御式编程引入的额外代码增加了软件的复杂度，反而容易造成错误
- ◆ 因此，只在适当的地方使用防御式编程

第3章 软件构建

- 3.1编码概述
 - 3.2编程语言和编码规范
 - 3.3可以工作的类
 - 3.4代码版本控制
 - 3.5高质量的子程序
 - 3.6防御式编程
 - 3.7变量和语句的构建
-

3.7 变量和语句的构建

- 使用变量的一般事项
- 变量名的力量
- 基本数据类型
- 组织直线型代码
- 使用条件语句
- 控制循环

使用变量的一般事项

- 数据认知
- 创建有效数据的第一步是了解所要创建的数据种类。积累大量的数据类型对于程序员来说至关重要。

使用变量的一般事项

■ 轻松掌握变量定义

- 关闭隐式声明
- 声明全部变量
- 遵循某种命名规则
- 检查变量名

使用变量的一般事项

■ 变量初始化原则:避免产生初始化错误的建议

- 在声明变量的时候初始化;
- 在靠近变量第一次使用的位置初始化;
- 理想情况下, 在靠近第一次使用变量的位置声明和定义该变量;
- 在可能的情况下使用**final**或者**const**;
- 特别注意计数器和累加器;
- 在类的构造函数里初始化该类的数据成员;

Visual Basic示例：糟糕的初始化

```
' declare all variables
Dim accountIndex As Integer
Dim total As Double
Dim done As Boolean

' initialize all variables
accountIndex = 0
total = 0.0
done = False
...

' code using accountIndex
...

' code using total
...

' code using done
While Not done
...

```

total在靠近首次使用的地方被声明和初始化。

done在靠近首次使用的地方被声明和初始化。

Visual Basic示例：良好的初始化

```
Dim accountIndex As Integer
accountIndex = 0
' code using accountIndex
...

Dim total As Double
total = 0.0
' code using total
...

Dim done As Boolean
done = False
' code using done
While Not done

```

使用变量的一般事项

- **变量初始化原则:**避免产生初始化错误的建议
 - 检查是否需要重新初始化;
 - 一次性初始化具名常量, 用可执行代码来初始化变量
 - 使用编译器设置来自动初始化所有变量;
 - 利用编译器的警告信息;
 - 检查输入参数的合法性;
 - 使用内存访问检查工具来检查错误的指针;
 - 程序开始时初始化工作内存。

使用变量的一般事项

- **作用域：**作用域或者可见性指的是变量在程序内的可见和可引用范围。
 - 使变量引用局部化；
 - 尽可能缩短变量的“存活”时间；
 - 减小作用域的一般原则；
 - 在循环开始之前再去初始化该循环里使用的变量，而不是在该循环所属的子程序的开始处初始化这些变量；
 - 直到变量即将被使用时再为其赋值；
 - 把相关语句放到一起；
 - 把相关语句提取成单独的子程序；
 - 开始时采用最严格的可见性，然后根据需要扩展变量的作用域。

使用变量的一般事项

■ 绑定时间

- 绑定时间是把变量和它的值绑定在一起的时间。是发生在编写代码的时候还是编译程序的时候？是在程序加载的时候还是在运行的时候？
 - 编码时（使用神秘数值）
 - 编译时（使用具名常量）
 - 加载时（从**Java**属性文件等外部数据源读取数据）
 - 对象实例化时（例如在每次窗体创建时读取数据）
 - 即时（例如在每次窗体重绘时读取数据）

Java示例：在编译时绑定其值的变量

```
private static final int COLOR_BLUE = 0xFF;  
private static final int TITLE_BAR_COLOR = COLOR_BLUE;  
...  
titleBar.color = TITLE_BAR_COLOR;
```

Java示例：在运行时绑定其值的变量

```
titleBar.color = ReadTitleBarColor();
```

一般而言，绑定时间越早灵活性就会越差，但复杂度也会降低。

使用变量的一般事项

- 数据类型和控制结构之间的关系
- 数据类型和控制结构之间以一种定义明确的方式相结合。**Jackson**描述出了三种类型的数据和相应的控制结构之间的关系：
 - 顺序型数据翻译为程序中的顺序结构；
 - 选择型数据翻译为程序中的**if**和**else**语句；
 - 迭代型数据翻译成程序中的**for**、**repeat**、**while**等循环结构。

使用变量的一般事项

- 为变量指定单一用途
- 通过使用一些巧妙的方法，可以给一个变量赋予多种职责。不过你最好还是远离这些技巧。
 - 每个变量只用于单一用途；
 - 避免让代码具有隐含含义；
 - 确保使用了所有以声明的变量。

使用变量的一般事项

■ 初始化变量（核对表）

- [] 每一个子程序都检查其输入参数的正确性吗？
- [] 变量声明位置靠近第一次使用的位置吗？
- [] 尽可能在声明变量的同时初始化变量吗？
- [] 如果无法同时声明和初始化变量，有没有在靠近第一次使用变量的位置声明变量？
- [] 计数器和累加器经过了适当的初始化吗？如果需要再一次使用，之前重新初始化了吗？
- [] 适当地重新初始化”需要重复执行的代码里的变量“了吗？
- [] 代码在通过编译器编译的时候是不是没有警告信息？
- [] 如果你用的语言允许隐式声明，你为由此可能引发的问题做好补偿措施了吗？

使用变量的一般事项

■ 使用变量的其它事项（核对表）

- [] 如果可能，所有变量都被定义为具有最小作用域了吗？
- [] 各变量的引用点都尽可能集中在一点吗？对同一变量的再次相邻引用，或者变量的整个生命期都这样做了吗？
- [] 控制结构符合数据类型吗？
- [] 所有声明的变量都用到了吗？
- [] 变量在合适的时间绑定了吗？——也就是说，你有意识地在晚期绑定所带来的灵活性和增加的复杂度之间做出了平衡了吗？
- [] 每个变量都有且仅有一项用途吗？
- [] 每个变量的含义都很明确且没有隐含含义吗？

使用变量的一般事项

■ 要点

- 数据初始化过程很容易出错，所以请用本章描述的初始化方法来避免由于非预期的初始值造成的错误；
- 最小化每个变量的作用域。把同一变量的引用点集中在一起。把变量限定在子程序或类的范围内。避免使用全局数据；
- 把使用相同变量的语句尽可能集中在一起；
- 早期绑定会减低灵活性，但有助于减小复杂度。晚期绑定可以增加灵活性，同时增加复杂度；
- 把每个变量用于唯一的用途。

变量名的力量

■ 选择好变量名的注意事项

- 一个好的变量名是可读的、易记得和恰如其分的。

■ 最重要的命名注意事项

- 最重要的考虑事项是，该名字要完全、准确地描述出该变量所代表的事物。通常，对变量的描述就是最佳的变量名。

■ 以问题为导向

- 一个好记的名字反映的通常是问题，而不是解决方案。

Java示例：糟糕的变量名

```
x = x - xx;  
xxx = fido + SalesTax( fido );  
x = x + LateFee( x1, x ) + xxx;  
x = x + Interest( x1, x );
```

Java示例：良好的变量名

```
balance = balance - lastPayment;  
monthlyTotal = newPurchases + SalesTax( newPurchases );  
balance = balance + LateFee( customerID, balance ) + monthlyTotal;  
balance = balance + Interest( customerID, balance );
```

变量名的力量

■ 最适当的名字长度

- 当变量名的平均长度在**10到16**个字符的时候，调试程序所需花费的力气是最小的。平均名字长度在**8到20**个字符的程序也容易调试。

■ 变量名对作用域的影响

- 较长的名字适用于很少用到的变量或者全局变量，而较短的名字则适用于局部变量或者循环变量。对于全局命名空间中的名字加限定词。

变量名的力量

■ 变量名中的计算值限定词

- 很多程序都有表示计算结果的变量：总额、平均值、最大值、等等。如果你要用类似于**Total**、**Sum**、**Average**、**Max**、**Min**、**Record**、**String**、**Pointer**这样的限定词来修改某个名字，那么请记住把限定词加到名字的最后。

变量名的力量

■ 变量名中的常用对仗词

- 通过应用命名规则来提高对仗词使用的一致性，常用的对仗词：

- begin/end
- first/last
- locked/unlocked
- min/max
- next/previous
- old/new
- opened/closed
- visible/invisible
- source/target

变量名的力量

- 为特定类型的数据命名
- 本节将讲述与循环变量、状态变量、临时变量、布尔变量、枚举类型和具名常量有关的考虑事项。
 - 为循环下标命名；
 - 为状态变量命名；
 - 为临时变量命名；
 - 为布尔变量命名；
 - 为枚举类型命名；
 - 为常量命名；

变量名的力量

■ 为什么要有规则

- 命名规则可以带来如下好处：
 - 要求你更多地按规矩办事；
 - 有助于在项目之间传递知识；
 - 有助于你在新项目中更快速地学习代码；
 - 有助于减少名字增生；
 - 弥补编程语言的不足之处；
 - 强调相关变量之间的关系。

变量名的力量

■ 何时采用命名规则

- 当多个程序员合作开发一个项目时；
- 当你计划把一个程序转交给另一位程序员来修改和维护的时候；
- 当你在组织中的其他程序员评估你写的程序的时候；
- 当你写的程序规模太大，以至于你无法在脑海里同时了解事情的全貌，而必须分而治之的时候；
- 当你写的程序生命周期足够长，长到你可能会在把它搁置几个星期或几个月之后又重新启动有关程序的工作时；
- 当你在一个项目中存在一些不常见的术语，并且希望在编写代码阶段使用标准的数据或者缩写的时候。

变量名的力量

- 与语言相关的命名规则的指导原则
- C的命名规则
 - c和ch是字符变量;
 - i和j是整数下标;
 - n表示某物的数量;
 - p是指针;
 - s是字符串;
 - 预处理宏全部大写;
 - 变量名和子程序名全部小写;
 - 下划线用作分隔符。

变量名的力量

■ 标准前缀

- 用户自定关于类型缩写
- 语义前缀
- 标准前缀的优点
 - 标准前缀能够更为精确地描述一些含义比较模糊的名字；
 - 标准化前缀是名字更加紧凑；
 - 标准前缀能帮助你准确地最类型做出判断。

变量名的力量

■ 创建具备可读性的短名字

■ 缩写的一般指导原则

- 使用标准的缩写；
- 去掉所有的非前置元音；
- 去掉虚词；
- 使用每一个单词的第一个或前几个字符、母；
- 统一地在每个单词的第一、第二或者第三个字母后截断；
- 保留每个单词的第一个和最后一个字母；
- 使用名字中每一个重要单词，最多不超多三个；
- 去除无用的后缀；

- ▣ 确保每一个音节中最引人注意的发音；
- ▣ 确保不要改变变量的含义；
- ▣ 反复使用上述技术，直到你把每个变量名的长度缩减到**8到20**个字符。

变量名的力量

■ 有关缩写的评论

- 不要从每个单词中删除一个字符的方式来缩写；
- 缩写要一致；
- 创建你能读出来的名字；
- 避免使用容易看错或者读错的字符组合；
- 使用词典来解决命名冲突；
- 在一份项目级的”标准缩写“文档中说明所有的缩写；
- 记住，名字对于代码读者的意义要比对作者更重要。

变量名的力量

■ 应该避免的名字指导原则

- ❑ 避免使用令人误解的名字或缩写；
- ❑ 避免使用具有相似含义的名字；
- ❑ 避免使用具有不同含义却具有相似名字的变量；
- ❑ 避免使用发音相近的名字；
- ❑ 避免在名字中使用数字；
- ❑ 避免在名字中拼错单词；
- ❑ 不要仅靠大小写来区分变量名；
- ❑ 避免使用多种自然语言；
- ❑ 避免使用标准类型、变量和子程序的名字；
- ❑ 不要使用与变量含义完全无关的名字；
- ❑ 避免在名字中包含易混淆的字符。

变量名的力量

■ 命名的一般注意事项（核对表）

- [] 名字完整并准确表达了变量所代表的含义了吗？
- [] 名字反映了现实世界的问题而不是编程语言方案吗？
- [] 名字足够长，可以让你无需苦苦思索吗？
- [] 如果有计算值限定符，它被放在了名字的最后吗？
- [] 名字中用Count或者Index来代替Num了吗？

变量名的力量

■ 为特定类型的数据命名（核对表）

- [] 循环下标的名字有意义吗？
- [] 所有的”临时“变量都重新命名以更有意义的名字了吗？
- [] 当布尔变量的值为真时，变量名能准确表达其含义吗？
- [] 枚举类型的名字中含有能够表示其类别的前缀或者后缀了吗？
- [] 具名长了是根据它所代表的抽象实体而不是它所代表的数字来命名的吗？

变量名的力量

■ 命名规则（核对表）

- [] 规则能够区分局部数据、类的数据和全局数据吗？
- [] 规则能够区分类型名、具名常量、枚举类型和变量名吗？
- [] 规则能够在编译器不强制检测只读参数的语言里标识出子程序中的输入参数吗？
- [] 规则尽可能地与语言的标准规则兼容吗？
- [] 名字为了可读性而加以格式化吗？

变量名的力量

■ 短名字（核对表）

- [] 代码使用了长名字吗？
- [] 是否避免只为了省一个字符而缩写名字的情况？
- [] 所有单词的缩写方式都一致吗？
- [] 名字能够读出来吗？
- [] 避免使用容易被看错或者读错的名字吗？
- [] 在缩写对照里对短名字做出说明了吗？

变量名的力量

■ 要点

- 好的变量名是提高程序可读性的一项关键要素；
- 名字要尽可能地具体；
- 命名规则应该能够区分局部数据、类数据和全局数据；
- 无论做哪种类型项目，你都应该采用某种变量命名规则；
- 现代编程语言很少要用到缩写；
- 代码阅读的次数远远多于编写的次数。

基本数据类型

■ 数值概论

- 避免使用神秘数值
- 预防除零错误；
- 使类型转换变得明显；
- 避免混合类型的比较；
- 注意编译器的警告

基本数据类型

■ 整数

- 检查整数除法；
- 检查整数溢出；
- 检查中间结果溢出。

■ 浮点数

- 避免数量级相差巨大的数之间的加减运算；
- 避免等量判断；
- 处理舍入问题：
 - 换用一种精度更高的变量类型；
 - 换用二进制编码的十进制变量；
 - 把浮点变量变成整数变量；

- 检查语言和函数库对特定数据类型的支持。

基本数据类型

■ 布尔变量

- 用布尔变量对程序加以文档说明；
- 用布尔变量来简化复杂的判断；
- 如果需要的话，创建你自己的布尔类型。

基本数据类型

■ 数组

- 数组是最简单和最常用的结构化数据类型。
- 确认所有的数组下标都没有超出数组边界；
- 考虑用容器来取代数组，或者将数组作为顺序化结构来处理；
- 检查数组的边界点；
- 如果数组是多维的，确认下标的使用顺序是正确的；
- 提防下标串话；

基本数据类型

■ 全局数据

- 全局数据可以在程序的任何位置访问。使用全局数据风险比使用局部数据大，通过一些子程序来访问数据很有用。

■ 与全局数据有关的常见问题

- 无意间修改了全局数据；
- 与全局数据有关的奇异的和令人激动的问题；
- 与全局数据有关的代码重入问题；
- 全局数据阻碍代码重用；
- 与全局数据有关的非确定的初始化顺序事宜；
- 全局数据破坏了模块化和智力上的可管理性。

基本数据类型

■ 使用全局数据的理由

- 保存全局数据；
- 模拟具名常量；
- 模拟枚举类型；
- 简化对其常用的数据的使用；
- 消除流浪数据。

■ 只有万不得已时才使用全局数据

- 首先把每一个变量设置为局部的，仅当需要时才把变量设置为全局的；
- 区分全局变量和类变量；
- 使用访问器子程序。

基本数据类型

- 用访问器子程序来取代全局数据
- 使用访问器子程序是实现抽象数据类型和信息隐藏的一种核心方法。
- 使用访问器子程序的优势；
 - 你获得了对数据的集中控制；
 - 你可以确保对变量的所有引用都得到了保护；
 - 你可以自动获得信息隐藏的普遍益处；
 - 访问器子程序可以很容易地转变为抽象数据类型。

组织直线型代码

- 必须有明确顺序的语句
- 如果语句之间存在依赖关系，并且这些关系要求你把语句按照一定的顺序加以排列，那么请设法使得这些关系变得明显。
 - 设法组织代码，使依赖关系变得非常明显；
 - 使子程序名能凸显依赖关系；
 - 利用子程序参数明确显示依赖关系；
 - 用注释对不清晰的依赖关系进行说明；
 - 用断言或者错误处理代码来检查依赖关系。

组织直线型代码

■ 顺序无关的语句

- 使代码易于自上而下地阅读；
- 把相关语句组织在一起。

使用条件语句

■ **if**语句

■ 简单**if-then**语句

- 首先写正常代码路径，再处理不常见情况；
- 确保对于等量的分支是正确的；
- 把正常情况的处理放在**if**后面而不要放在**else**后面；
- 让**if**子句后面跟随一个有意义的语句；
- 考虑**else**子句；
- 测试**else**子句的正确性；
- 检查**if**和**else**子句是不是弄反了。

Visual Basic示例：随意地处理大量错误的代码

```
OpenFile( inputFile, status )  
If ( status = Status_Error ) Then  
    errorType = FileOpenError  
Else  
    ReadFile( inputFile, fileData, status )  
    If ( status = Status_Success ) Then  
        SummarizeFileData( fileData, summaryData, status )  
        If ( status = Status_Error ) Then  
            errorType = ErrorType_DataSummaryError  
        Else  
            PrintSummary( summaryData )  
            SaveSummaryData( summaryData, status )  
            If ( status = Status_Error ) Then  
                errorType = ErrorType_SummarySaveError  
            Else  
                UpdateAllAccounts()  
                EraseUndoFile()  
                errorType = ErrorType_None  
            End If  
        End If  
    Else  
        errorType = ErrorType_FileReadError  
    End If  
End If
```

出错情况。

正常情况。

正常情况。

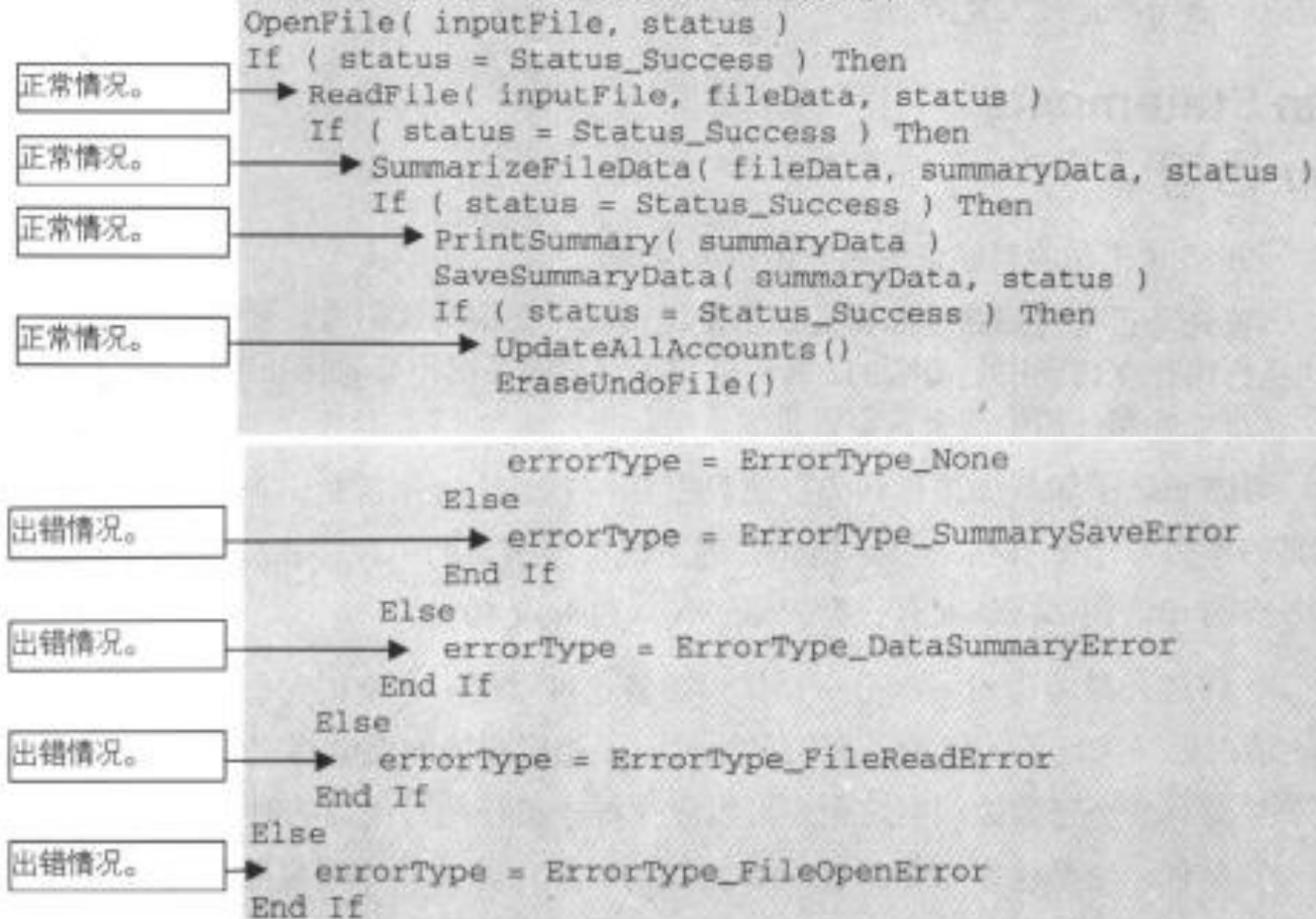
出错情况。

正常情况。

出错情况。

正常情况。

Visual Basic示例：系统地处理大量错误的代码



使用条件语句

■ **if-then-else**语句串

- 利用布尔函数调用简化负载的检测；
- 把最常见的情况放在最前面；
- 确保所有情况都考虑到了；

使用条件语句

- **case**语句
- 为**case**选择最有效的排列顺序
 - 按字母顺序或按数字顺序排列各种情况；
 - 把正常的情况放在前面；
 - 按执行频率排列**case**语句。

使用条件语句

■ 使用**case**语句的诀窍

- 简化每种情况对应的操作；
- 不要为了使用**case**语句而可以制造一个变量；
- 把**default**子句只用于检查真正的默认情况；
- 利用**default**子句来检测错误；
- 在C++和Java里，避免代码执行越过一条子句的末尾；
- 在C++里，在**case**末尾明确无误地标明需要穿越执行的程序流程。

控制循环

■ 选择循环的种类

- 在大多数语言中，你只能用到少数几个循环：
- 计数循环执行的次数是一定的，可能是针对每位雇员执行一次；
- 连续求值的循环预先并不知道将要执行多少次，它会在每次迭代时检查是否应该结束；
- 无限循环一旦启动就会一直执行下去；
- 迭代器循环对容器内的每个元素执行一次操作。

控制循环

■ 什么时候使用**while**循环

- 如果你预先并不知道循环要迭代多少次，那么就使用**while**循环。
- 检测位于循环的开始；
- 检测位于循环的结尾。

■ 什么时候用带退出的循环

- 带退出的循环就是终止条件出现在循环中间而不是开始或者末尾的循环。
- 正常的带推出循环；
- 非正常的带推出循环。

控制循环

■ 何时使用**for**循环

- 如果你需要一个执行次数固定的循环，那么**for**循环就是一个很好的选择。

■ 何时使用**for each**循环

- **for each**循环或其等价物很适用于对数组或者其他容器的各项元素执行操作。

控制循环

■ 循环控制

- 进入循环：
 - 只从一个位置进入循环；
 - 把初始化代码紧放在循环前面；
 - 用**while(true)**表示无限循环；
 - 在适当的情况下多使用**for**循环；
 - 在**while**循环更适用时候，不要使用**for**循环。
- 处理好循环体：
 - 用“{”和“}”把循环中的语句括起来；
 - 避免空循环；
 - 把循环内务操作要么放在循环的开始，要么放在循环的末尾；
- 一个循环只做一件事。

控制循环

□ 退出循环：

- 设法确认循环能够终止；
- 使循环终止条件看起来很明显；
- 不要为了终止循环而胡乱改动**for**循环的下标；
- 避免出现依赖于循环下标最终取值的代码；
- 考虑使用安全计数器。

□ 提前退出循环：

- 考虑在**while**循环中使用**break**语句而不用布尔标记；
- 小心那些有很多**break**散布其中的循环；
- 在循环开始处用**continue**进行判断；
- 如果语言支持，请使用带标号的**break**结构；
- 使用**break**和**continue**时要小心谨慎。

控制循环

□ 使用循环变量：

- 用整数或者枚举类型表示数组和循环的边界；
- 在嵌套循环中使用有意义的变量名来提高可读性；
- 用有意义的名字来避免下标串话；
- 把循环下标变量的作用域限制在本循环内。

□ 循环长度：

- 循环要尽可能地短，以便能够一目了然；
- 把嵌套限制在**3**层以内；
- 把长循环的内容移到子程序里；
- 要让长循环格外清晰。

Java示例：差劲的循环变量名

```
for ( int i = 0; i < numPayCodes; i++ ) {  
    for ( int j = 0; j < 12; j++ ) {  
        for ( int k = 0; k < numDivisions; k++ ) {  
            sum = sum + transaction[ j ][ i ][ k ];  
        }  
    }  
}
```

Java示例：好的循环变量名

```
for ( int payCodeIdx = 0; payCodeIdx < numPayCodes; payCodeIdx++ ){  
    for (int month = 0; month < 12; month++ ) {  
        for ( int divisionIdx = 0;  
            divisionIdx < numDivisions;  
            divisionIdx++ ){  
            sum = sum + transaction[ month ][ payCodeIdx ][ divisionIdx ];  
        }  
    }  
}
```

C++示例：下标串话

在这里首次用到。

```
for ( i = 0; i < numPayCodes; i++ ) {  
    // lots of code  
    ...  
    for ( j = 0; j < 12; j++ ) {  
        // lots of code
```

在这里又出现了。

```
    ...  
    for ( i = 0; i < numDivisions; i++ ) {  
        sum = sum + transaction[ j ][ i ][ k ];  
    }  
}  
}
```